KOREA AI IN ACTION · SESSION 2

From Sequence Models to Transformers

— and how we steer LLMs with prompts

A 3-hour intuitive tour · no math, no code — just the story

TODAY'S ROADMAP

Three hours, one story

Part 1 · ~75 min

From Sequence Models to Transformers

How AI learned to read — one word at a time, forgetting, attention, and the breakthrough behind today's chatbots

Break · 15 min

Coffee & questions

Stretch, refill, ask anything from Part 1

Part 2 · ~75 min

Prompting & How LLMs Are Steered

Zero-shot vs. few-shot, roles and context, the dials you can turn — and what an API actually is

PART 1

From Sequence Models to Transformers

How machines learned to read — and why they kept forgetting

BEFORE LANGUAGE

The world runs on sequences

A sequence is any data where order carries information — and predicting “what comes next” is one of the oldest problems in AI:

Stock prices

Yesterday's moves shape today's forecast — order is everything.

Weather

Tomorrow's forecast is a continuation of the last few days' pattern.

Speech

Sound arrives as a stream; each syllable only makes sense after the last.

Music

A melody is notes in time — shuffle them and the song is gone.

DNA & proteins

Life itself is written as a sequence of letters, read in order.

Language

Words in order — and the richest, messiest sequence of them all.

Same family of problems, one shared question: given what came before, what comes next?

THE COMMON THREAD

Different data, same three challenges

Order is meaning

Reversing a stock chart, a melody, or a sentence destroys it. A model must respect sequence — it can't treat the data as a loose bag of items.

Predict what's next

Next price, next note, next syllable — almost every sequence task boils down to continuing the series from its past.

The past can be long

Sometimes the crucial clue sits far behind you: a market crash last quarter, a theme from the song's opening bars, a name mentioned two paragraphs ago.

The richest, messiest sequence of all is human language — *so that's where the story goes next.*

THE STARTING POINT

Language is a sequence — order is meaning

“Dog bites man.”

Ordinary Tuesday.

“Man bites dog.”

Front-page news.

Same three words. Completely different meaning.

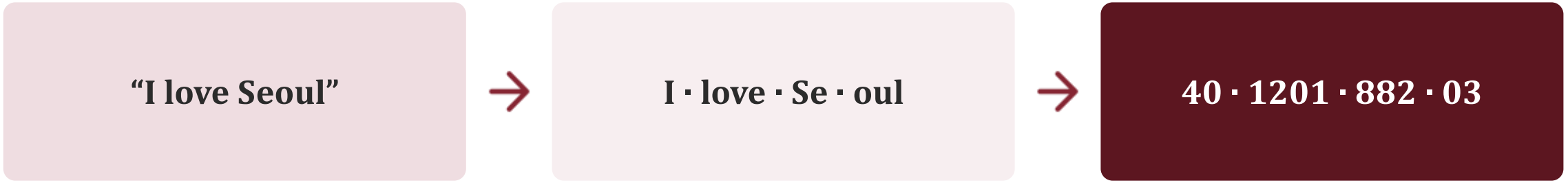
So any AI that handles language can't just look at a bag of words — it must process them in order. That single requirement shaped 30 years of research.

FIRST HURDLE

One problem before anything else: computers can't read

01
10

A computer is a number-crunching machine. To it, “hello” is not a greeting — it's nothing at all, until we turn it into numbers. So every language AI, from your phone's keyboard to Claude, starts with the same two-step trick:



Raw text

Step 1 — chop into pieces

Step 2 — number each piece

Those pieces have a name you'll hear constantly around AI: tokens.

What exactly is a token?

A token is one piece from the model's fixed puzzle set — often a whole short word, but frequently just a chunk of one. Common words stay in one piece; longer or rarer words get assembled from parts:

"Transformers are unbelievable!" → 7 tokens. Word chunks, whole words, and punctuation each count.

Rule of thumb (English): 1 token ≈ ¾ of a word, so 1,000 tokens ≈ 750 words — about one page of text. The model never sees your letters or words directly; it sees only this stream of tokens.

A GOLDILOCKS CHOICE

Why pieces — not whole words, not single letters?

Whole words? Too big

You'd need millions of entries — every name, typo, and brand-new word. And the first unknown word (“skibidi”?) breaks the system completely.

Single letters? Too small

Only ~100 symbols needed — but every sentence becomes enormously long, and each letter carries almost no meaning on its own.

Word pieces? Just right

A few tens of thousands of pieces can spell anything ever written. Common words stay whole; rare or new words get built from parts, like Lego.

This trade-off — coverage vs. length — is why token lists look strange to humans: they're optimized for machines, learned from real text frequency.

NOT ALL LANGUAGES ARE EQUAL

The same sentence costs different tokens

Token inventories are learned mostly from web text — which skews heavily English. Languages with less training text get chopped into finer pieces, so the same meaning often takes noticeably more tokens:

“I love you” (English)

typically ~3 tokens

“사랑해” (Korean)

often more tokens — despite fewer characters

Why should you care? More tokens = higher API cost, slower responses, and a context window that fills up faster. Newer models keep improving their Korean tokenization — but the asymmetry hasn't fully disappeared.

MINI ACTIVITY · 5 MIN

Guess the tokens

With your neighbor: rank these from fewest to most tokens. Then we'll check live with an online tokenizer.

A `cat`

B `cats!!!`

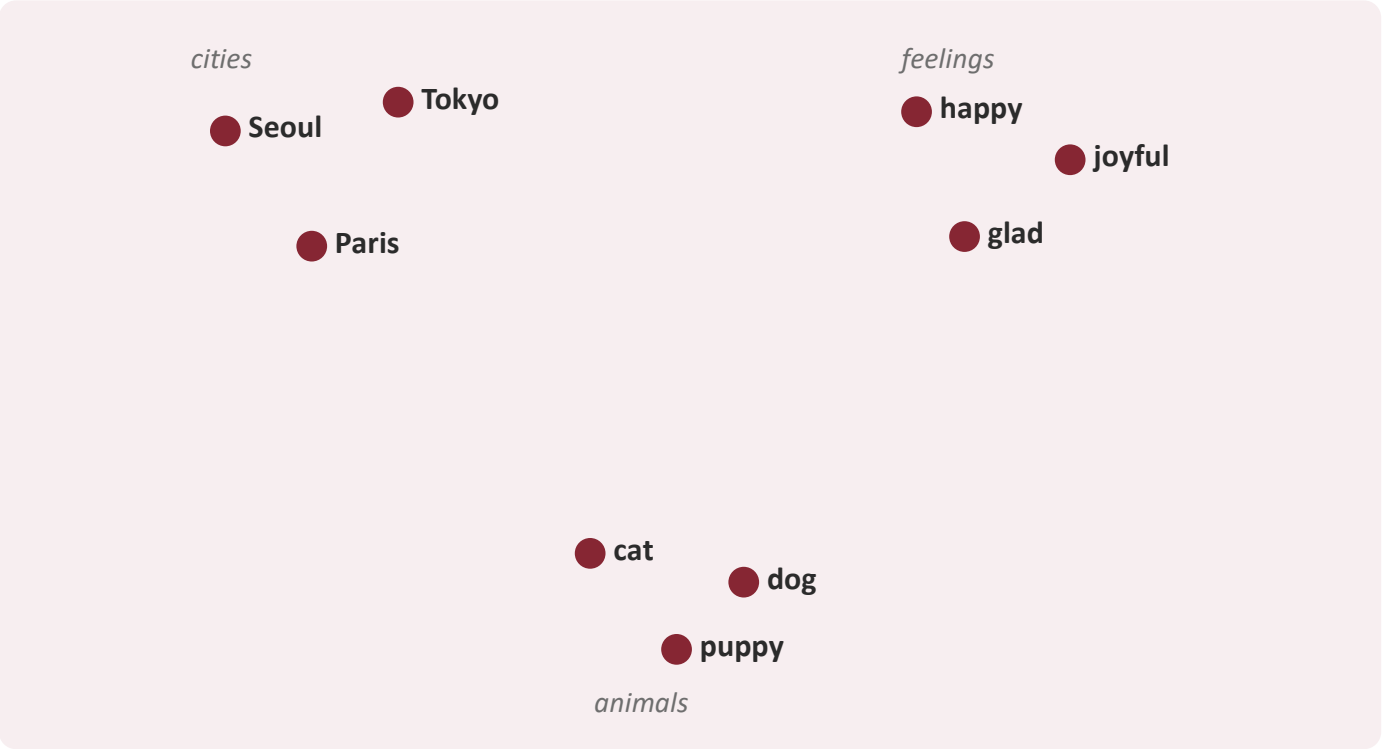
C `고양이`

D `supercalifragilisticexpialidocious`

Bonus question: why might “cats!!!” cost more tokens than “cat” — and what do you predict for the emoji 😼?

Every token gets a home on a map of meaning

A token ID is just a label. The model's real magic: it learns to place every token as a point on a giant internal map — where distance means similarity.



How is the map drawn?

Purely from company kept: words that appear in similar sentences drift together. Nobody tells the model “Seoul is a city” — it reads millions of sentences where Seoul behaves like Tokyo and Paris.

Four places tokens will affect you directly

Pricing

AI services bill by the token, in and out. Verbose prompts and long documents literally cost more money.

Limits

A model's “context window” — how much it can consider at once — is measured in tokens. Long chats and big files hit this ceiling.

Speed

Replies are generated token by token — that's the typing effect, and why long answers take longer.

Quirks

“How many r's in strawberry?” trips models up — they see tokens like “straw”+“berry”, not letters. Counting characters is genuinely hard for them.

Keep “tokens” in your pocket — it comes back in Part 2 when we talk about context windows.

BEFORE NEURAL NETWORKS

You've been using a language model for years

Your phone keyboard's word suggestions are a tiny language model. The classic version is beautifully dumb: look at the last one or two words, and suggest whatever most often follows them in a big table of counted phrases.

Where it shines

"See you → soon / later / tomorrow." Two words of memory are plenty for everyday phrases.

Where it collapses

"The girl who grew up in Paris … speaks fluent → ???" Two words of memory can't reach back to "Paris." It's not even trying.

The obvious dream: a model that remembers the whole sentence, not just the last two words. Enter neural networks.

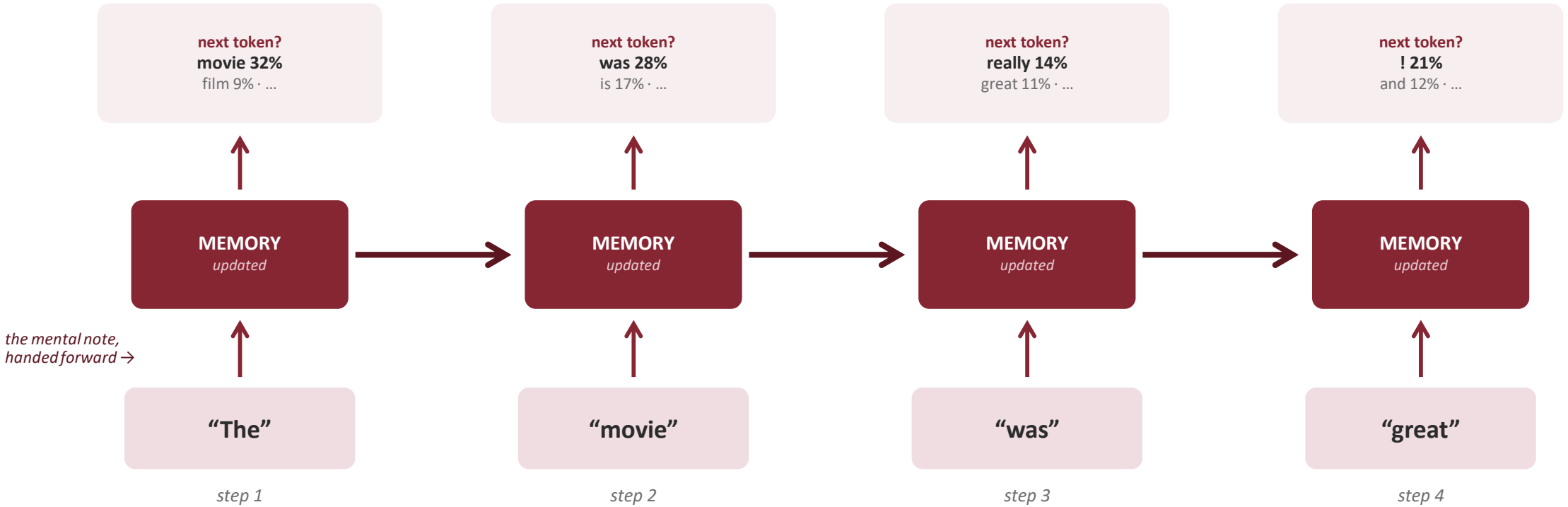
Reading through a keyhole, one word at a time

A Recurrent Neural Network (RNN) reads exactly the way the arrows suggest: one word, then the next, carrying a running summary of everything so far — like reading a book through a keyhole that shows one word at a time.

The key idea: memory. After each word, the model updates one internal "mental note" that must summarize the entire sentence so far. Every new word overwrites a little of that note.

THE SAME IDEA, DRAWN OUT

Inside an RNN: one step, then the next

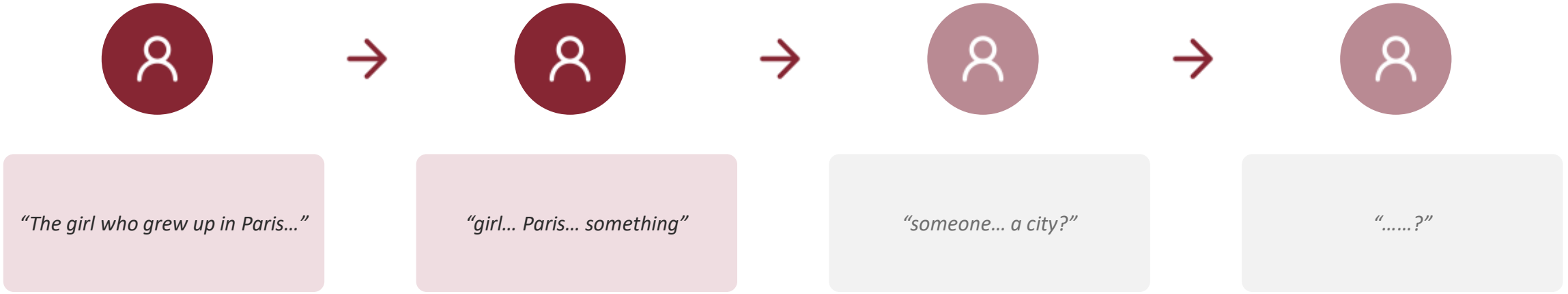


Each step: ① a token comes in → ② the memory updates, carrying everything so far → ③ the model outputs a probability for every possible next token. Then — and only then — the next step may begin.

“Sequential” means exactly this waiting line — and remember the top row: it returns as THE training game.

THE HIDDEN WEAKNESS

Memory as a game of telephone



Each step passes along a compressed summary — and every hand-off loses a little detail. By the end of a long sentence, the beginning has faded into a rumor.

This is exactly what happens inside an RNN as sentences get longer.

WHY IT MATTERS

Language often depends on faraway words

"**The girl** who grew up in Paris, moved to Seoul for university, and studied engineering for six years, speaks fluent ____."

You

You instantly say "French" — your eyes jump back to "Paris" without effort.

An RNN

By the time it reaches the blank, "Paris" was 20 words ago — the memory of it has mostly been overwritten.

A notebook with rules: write, erase, read

Long Short-Term Memory networks gave the reader a structured notebook instead of one fading mental note. Little “gates” decide, at every word:

Write

Is this word important enough to note down? (“Paris” → yes)

Erase

Is an old note no longer needed? Clear space for what matters.

Read

Which notes are relevant right now? (blank after “speaks fluent” → check “Paris”)

A real improvement — LSTMs powered Google Translate and Siri for years. But notice what didn't change: still reading one word at a time, still squeezing everything into one notebook.

2014–2017

The LSTM golden age: sequence models go mainstream

Google Translate

Around 2016 Translate switched to neural sequence models — users woke up to translations that suddenly read like sentences, not word salad.

Voice assistants

Siri, Alexa, and Google Assistant leaned on sequence models to turn your speech stream into text and intent.

Smart Reply

Gmail began suggesting whole responses — “Sounds good!” “See you then!” — an early taste of AI writing for you.

Real products, real users, real value — sequence models had arrived. And yet, behind the scenes, engineers kept fighting the same enemy: long inputs.

THE CLEAREST SYMPTOM

Translation exposed the bottleneck

Neural translation worked in two halves: one network reads the whole source sentence and squeezes it into a single fixed-size summary; a second network unfolds the translation from that summary alone.

Imagine the analog version: watch a two-hour movie, write ONE sentence about it, hand that sentence to a friend — and ask them to re-tell the entire movie. Short films? Fine. Long ones? Details are simply gone.

Researchers asked the obvious question: while writing the translation, why can't we peek back at the original?

Two problems no notebook could fix

1 · Sequential = slow

You cannot read word 50 before word 49. No shortcuts, no parallel reading — so training on huge amounts of text took forever. Bigger models were simply impractical.

2 · One bottleneck of memory

However clever the gates, the whole past still had to fit through a single fixed-size summary. Long documents, subtle references, and far-apart connections kept slipping away.

By 2016, progress was flattening. The field needed a different way to read.

DISCUSSION · 10 MIN

How do you actually read?

“The trophy didn't fit in the suitcase because it was too big.” — What was too big?

1. How did you resolve “it”? Did you re-read, or did your eyes jump straight to the right word?
2. Swap “big” for “small”. What does “it” mean now? What changed?
3. Discuss with your neighbor: if you could redesign the machine reader, what ability would you give it?

THE BREAKTHROUGH

Attention: stop summarizing — start looking back

The idea you just invented has a name: attention. Instead of relying on a fading summary, let the model keep every word on the table — and for each new word, let it decide which earlier words to “pay attention to.”

Think of a highlighter. When you answer a question about a page, you don't recite the whole page from memory — you glance back and highlight the two or three phrases that matter for this question. Attention is a learned highlighter: for every word it processes, the model highlights the relevant context, strongly or faintly, and blends what it finds.

No more telephone game. No more single bottleneck. Direct access to any word, however far away.

ATTENTION IN ACTION

Watching the model decide what “it” means

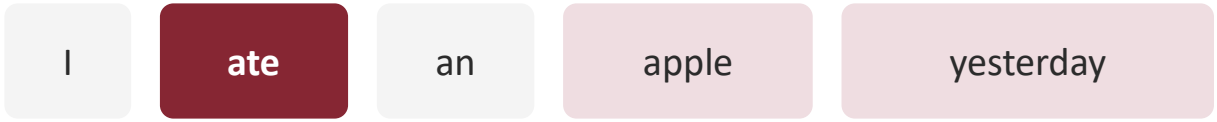
The	**animal**	didn't	cross	the	street	because	**it**	was	too	tired.

When the model processes **“it”**, attention lights up **“animal”** strongly and “street” faintly — because “tired” fits an animal, not a street. Change “tired” to “wide” and the highlight jumps to “street.”

Nobody programmed that rule. The model learned where to look purely from reading vast amounts of text — the same way you learned it: from experience, not from a grammar textbook.

Born in translation — Korean shows why

Attention first appeared (2014–15) as a fix for the translation bottleneck: while writing each output word, let the translator glance back at the original. Korean↔English makes the need obvious — the word order flips:



English source

Korean translation — the verb comes last

While writing "먹었다" — the LAST Korean word — the model must attend to "ate", the SECOND English word. No fixed left-to-right memory handles this gracefully. A learned "look-back" does it effortlessly — and translation quality jumped immediately.

ONE MORE UPGRADE

Not one highlighter — a whole pencil case

Real models run many attention “highlighters” at once, each trained to hunt for something different in the same sentence. Their findings are blended into one rich reading:

Who did what

links subjects to their verbs — “the girl … speaks”

References

resolves “it”, “she”, “the former” back to their owners

Time & place

tracks “yesterday”, “in Paris”, tense markers

Tone & register

notices formal vs. casual, sarcasm cues, politeness levels

(These labels are illustrative — the model invents its own specialties during training, and researchers discover them afterward by peeking inside.)

Multiple simultaneous perspectives on every sentence — that's the reading superpower we hand to the Transformer.

2017 — “ATTENTION IS ALL YOU NEED”

The Transformer: everyone looks at everyone, all at once

All words, simultaneously

Drop the left-to-right march. Every word attends to every other word in parallel — the whole sentence is processed at once, like a room full of readers comparing notes.

Parallel = trainable at scale

Because nothing waits in line, training runs beautifully on modern chips. Suddenly you could feed a model not a library shelf, but the internet.

Stack it deep

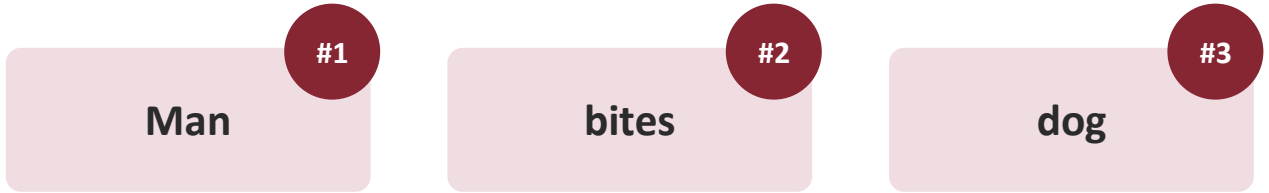
Layer attention on attention: early layers catch grammar, later layers catch meaning, references, even style. Depth turns a reading trick into understanding.

One 2017 paper replaced 20 years of sequential reading. Every model you'll hear about today — GPT, Claude, Gemini — is a Transformer.

A PUZZLE YOU MIGHT HAVE SPOTTED

Reading all at once… then how does it know order?

Sharp question: we spent half this session proving order is meaning — “Dog bites man” vs. “Man bites dog.” But if the Transformer reads all words simultaneously, doesn't the order disappear?



Every token boards with a seat number

The fix is almost embarrassingly simple: before reading, stamp each token with its position — a seat number that travels with it. “dog @ seat 3” and “dog @ seat 1” are now different things, so attention can use order without reading in order.

Have the best of both: parallel reading and a sense of sequence.

One simple game: guess the next word

"The capital of France is ___"

That's the entire training objective. Show the model trillions of words of real text, hide the next word, ask it to guess, correct it, repeat. A chatbot's reply is just this game played over and over — one word at a time, each guess shaped by attention over everything before it.

THE SURPRISE

Predicting well quietly requires understanding

To keep winning the guessing game across all of human text, the model is forced to absorb what the text depends on:

Grammar

“She ___” → a verb is coming — and it must agree with “she.”

Facts

“The capital of France is ___” → you can't guess Paris without knowing it.

Reasoning

“Tom is taller than Jane. Jane is taller than Sue. The tallest is ___” → the next word requires a logical step.

Style & tone

A legal contract and a K-drama script continue very differently — the model must feel the difference.

A “dumb” objective, pushed far enough, produces surprisingly smart behavior.

THE FLIP SIDE

The game rewards plausible — not true

Here's the catch hiding inside next-token prediction: the model is trained to continue text convincingly. Usually the convincing continuation is also the correct one. But when the model doesn't know, the most likely next tokens still arrive — fluent, confident, and wrong.

What it looks like

Invented paper citations that sound real. A confident wrong date. A plausible-looking statistic no one ever measured. Details polished, substance missing.

Why it happens

The model completes patterns; it doesn't consult a fact database before speaking. “According to a 2019 study…” is a very common pattern — so it flows out easily, study or no study.

The industry word is “hallucination.” It's improving fast — newer models decline more and search the web to check — but it will never be worth zero caution.

Hold this thought — in Part 2 we'll turn it into prompting habits.

Three ingredients turned a model into a chatbot

Scale

Thousands of times more text and vastly bigger models than the LSTM era. New abilities — translation, coding, summarizing — appeared without being explicitly taught.

Chat tuning

Raw predictors just continue text. Extra training with human feedback taught models to answer, follow instructions, and decline — turning autocomplete into a conversation partner.

Everything, one model

Older AI needed one system per task. An LLM handles email, code, poetry, and analysis with the same weights — the task is set by your words, not by new engineering.

"Large Language Model" = a Transformer, trained on the guessing game at enormous scale, then tuned to talk.

HOW BIG IS “LARGE”?

Reading the internet, literally

Trillions

of tokens of text — web pages, books, articles, code. More than a person could read in thousands of lifetimes.

Months

of non-stop training on warehouses full of specialized chips, playing the guessing game billions of times per second.

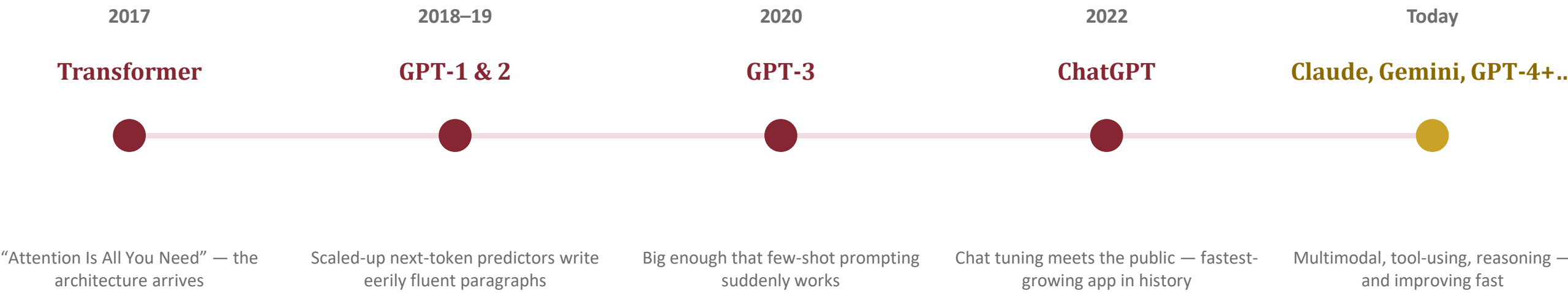
Billions+

of internal dials (“parameters”) adjusted along the way — the learned knowledge lives in their settings, not in any stored text.

Key mental model: the model doesn't store the internet — it distills patterns from it, the way you remember lessons from books you've long forgotten word-for-word.

FROM PAPER TO PHENOMENON

Eight years from one idea to your pocket



Notice what did NOT change: *every model on this line plays the same next-token game on the same Transformer idea. The years bought scale, tuning, and polish — not a new secret.*

HONEST LIMITS

Knowing the edges makes you a better user

Frozen in time

Training ended at some cutoff date. Without a web-search feature turned on, last week's news simply doesn't exist for it.

No memory by default

A fresh chat knows nothing about you or your previous chats (unless the app adds a memory feature on top). “As I said yesterday” means nothing.

Sees tokens, not letters

Character counting, spelling games, precise text surgery — the strawberry problem from earlier lives here.

Fluent ≠ correct

Confidence is a writing style, not a truth signal. The polish of an answer tells you nothing about its accuracy.

None of these are reasons not to use LLMs — they're the user manual nobody hands you.

SO WHEN YOU TYPE INTO CHATGPT, CLAUDE, OR GEMINI…

…you are watching everything from today at once

Attention reads your whole message — and the whole conversation — deciding which of your words matter for each word of the reply.

A Transformer processes it in parallel, which is why replies begin in a second, not a minute.

Next-token prediction writes the answer one word at a time — that's the typing effect you see on screen.

Scale + chat tuning are why one tool handles your email, your homework, and your code.

The tools you use daily are this story, shipped.

PART 1 RECAP

The road to the models you use

One sentence to keep: *AI stopped trying to remember language and learned to look at it — and that single change made everything else possible.*

QUICK CHECK · 10 MIN

Pop quiz — shout it out

Q1 Why does the model see “un + believ + able” instead of “unbelievable”?

Q2 What was the RNN's “telephone game” problem, in one sentence?

Q3 Attention replaced a fading summary with what ability?

Q4 Why did parallel reading matter so much for building bigger models?

Q5 What single game trains an LLM — and what's its dark side?

Break — 15 minutes

When we're back: you take the wheel — steering these models with prompts.

PART 2

Prompting & How LLMs Are Steered

The model is fixed. Your words are the steering wheel.

THE CORE IDEA

A brilliant improviser is waiting for a brief

An LLM is like a world-class improv actor: it can play any role, in any style, on any topic. But an improviser is only as good as the brief you hand them. “Do something!” gets a shrug; a clear scene gets a performance.

“Write about marketing.”

Vague brief → generic essay that helps no one.

“Write three subject lines for a re-engagement email to customers inactive for 90 days. Friendly, under 8 words each.”

Clear brief → output you can actually use.

Same model, same second — wildly different value. The difference was entirely in the prompt.

A SECRET WORTH KNOWING

You're never the only one prompting

Before your message ever reaches the model, the app quietly places its own instructions in front of it — a standing brief called a system prompt:

“You are a helpful assistant. Be accurate; admit uncertainty. Answer in the user's language. Refuse harmful requests. Keep formatting clean…” ← then your message arrives

- It's why the same underlying model feels different across apps — different standing briefs, different personalities.
- It's why a chatbot answers in Korean when you write in Korean — someone prompted that behavior into place.
- And it's proof of today's thesis: prompting isn't a user trick — it's how the professionals steer models too.

TECHNIQUE 1

Zero-shot: just ask

"Zero-shot" means asking with zero examples — the model relies purely on what it absorbed during training.

"Classify this customer review as positive, negative, or neutral: 'The delivery was fast but the box arrived damaged.'"

Works well when…

- The task is common (summarize, translate, classify, draft)
- Ordinary instructions describe it fully
- You'd trust a smart intern with just one sentence of guidance

Struggles when…

- You want a very specific format or house style
- The task has unusual, hard-to-verbalize rules
- "I'll know it when I see it" — you can't fully describe it

TECHNIQUE 2

Few-shot: show, don't tell

Give 2–5 examples of exactly what you want, then your real input. The model continues the pattern — often more faithfully than it follows any written rule.

Turn feature notes into release-note lines:

Notes: login now 2x faster
Line: ⚡ Sign in twice as fast — we tuned the engine.

Notes: fixed crash on photo upload
Line: 📸 Photo uploads are rock-solid again.

Notes: dark mode added
Line:

Why it works

The examples silently teach tone, emoji use, length, and structure — things that are tedious to describe but instant to demonstrate. The model attends to your examples and matches the pattern.

Rule of thumb: if explaining the format takes longer than showing it — show it.

TECHNIQUE 2½

“Show your steps” — ask for the working, not the answer

For anything with reasoning — math, logic, planning, tricky comparisons — don't ask for the answer. Ask for the working first, then the answer.

“I bought 3 notebooks at ₩2,700 each and paid ₩10,000. My change?”

Direct ask: the model may blurt a plausible-looking number — and plausible is exactly the failure mode we met in Part 1.

Same question + “Work through it step by step, then give the answer.”

Now: 3 × 2,700 = 8,100 → 10,000 − 8,100 = ₩1,900. Each written step is checked-able — by you AND by the model itself.

Why it works — a beautiful callback: the model generates token by token, and every step it writes becomes context its attention reads for the next step. Writing the reasoning literally gives it more to think with. (Modern “reasoning modes” automate exactly this.)

TECHNIQUE 3

Give the model a role

"You are a …" is not decoration. It shifts vocabulary, priorities, and what the model assumes you already know.

Same question: "Explain why my app is slow."

"You are a patient teacher for non-programmers."

Everyday analogies: "your app is a kitchen with one cook…" — no jargon.

"You are a senior performance engineer."

Systematic checklist: measurements to run first, likely bottlenecks, what to inspect in what order.

"You are a startup advisor."

Business lens: is speed actually losing you users? What's the cheapest fix that moves the metric?

TECHNIQUE 4

Context is fuel — the model can't read your mind

The model knows the internet but knows nothing about you, your audience, or your situation — unless you say it. Four things almost always worth stating:

Audience

"For first-year students with no coding background…"

Goal

"I'm trying to convince my manager to fund this…"

Background

"We already tried X and it failed because…"

Constraints

"Must fit one page; avoid technical terms; formal tone…"

A good test: could a talented stranger do this task with only your prompt? If not, add context.

The context window: the model's desk space

Everything the model considers — the app's hidden brief, your whole conversation, its own replies, any pasted documents — must fit on one desk at once. That desk is the context window, and it's measured in tokens.

What happens when the desk fills

The oldest papers slide off or get compressed. That's the real story behind “it forgot the instructions I gave an hour ago” — not moodiness, just a full desk.

Working with the desk, not against it

Restate key constraints in long chats · start a fresh chat per topic · paste the relevant pages, not the whole document · ask for a summary of the conversation so far, then continue from it.

Desk sizes have grown from a few pages to entire books' worth — but they are always finite, and long inputs still get skimmed harder than short ones.

Tone, length, format — just ask for it

Tone

“Warm and encouraging” vs. “blunt and critical” vs. “neutral and formal” — the model can do all of them, but defaults to polite-generic unless told.

Ask for: “write like a friendly senior colleague”

Length

Models tend to over-explain. Word counts, sentence limits, and “one paragraph” are all respected surprisingly well.

Ask for: “maximum 3 sentences”

Format

Table, bullet list, step-by-step, JSON, email draft — structure is free if you request it, and painful to fix if you don't.

Ask for: “a two-column table: risk / mitigation”

Temperature: the creativity dial

Remember: the model ranks possible next words. Temperature controls how adventurous the pick is.

Low temperature

Always take the safest word. Focused, consistent, repeatable — the same question gets nearly the same answer.

Best for: facts, summaries, analysis, anything where being right beats being interesting.

High temperature

Sometimes take a bold, lower-ranked word. Diverse, surprising, occasionally weird — the same question wanders.

Best for: brainstorming, naming, story ideas, breaking out of clichés.

In chat apps this dial is mostly preset; in API tools you set it yourself. Either way, now you know why the same prompt can give different answers twice.

THE HABIT THAT BEATS ANY TRICK

Your first prompt is a first draft

Nobody writes the perfect prompt on try one — and nobody needs to. The conversation itself is the interface: run, look, steer, run again.

1

Run

Fire off a decent first attempt — don't polish it.

2

Inspect

What's actually wrong? Too long? Wrong tone? Missed the point?

3

Steer

“Halve it.” · “Less formal.” · “Keep point 2, cut the rest.” · “More like your second example.”

4

Repeat

Two or three rounds usually beats one ‘perfect’ mega-prompt.

Steering commands can be tiny — the model attends to your feedback the same way it attends to everything else.

THE HALLUCINATION ANTIDOTE

Make the model its own first reviewer

Remember Part 1: fluent ≠ correct. You can't fix that with hope — but you can build checking into the prompt:

"List the assumptions you're making."	Surfaces the guesses hiding inside a confident answer.
"What could be wrong with this answer?"	A second pass in critic mode catches a surprising amount.
"Flag anything you're not sure about."	Invites the model to mark its own weak spots instead of smoothing them over.
"Critique this draft, then rewrite it."	Critic-then-editor beats single-pass generation for almost any writing task.

Honest limit: self-checking reduces errors — it does not eliminate them. Facts that matter still get verified outside the chat.

WHAT GOES WRONG

Four prompt anti-patterns (you'll recognize them)

The kitchen sink

Ten requests in one message → ten half-done answers. Split big jobs into steps; let each response feed the next.

The buried question

Your real ask hidden mid-paragraph gets diluted. Put the task first or last — the positions that stand out.

Assuming memory

“Like the report I mentioned” — a fresh chat has no idea. Restate or re-paste what matters. (The empty desk, remember?)

The vague quality bar

“Make it good” outsources your standards. Define good: for whom, how long, what tone, what to avoid.

Every anti-pattern breaks the same rule: the talented stranger couldn't act on it either.

PUTTING IT TOGETHER

The anatomy of a strong prompt

Role	*"You are an experienced HR manager…"*
Context	*"Our 20-person startup just missed its quarterly target…"*
Task	*"Draft a message to the team about the results…"*
Format & tone	*"Under 150 words, honest but motivating, no corporate clichés…"*
Examples (optional)	*"Here's a past message the team responded well to: …"*

Not every prompt needs all five — but when output disappoints, one of these is usually missing.

HANDS-ON · 25 MIN

Workshop: rescue a bad prompt

Starting prompt: “Write something about our new product.”

1 **Rewrite (10 min)**

In pairs: rebuild it using Role · Context · Task · Format · Examples. Invent any product you like.

2 **Run it (10 min)**

Try both versions in ChatGPT, Claude, or Gemini. Then change ONE element — the role, or the format — and run again.

3 **Share (5 min)**

Which single change moved the output most? Most groups are surprised by the answer.

ONE LAST PIECE

What's an API? The plug that lets apps talk to the model

A chat window is one door into the model — built for humans. An API is a second door — built for software. Same model behind both.

Think of a wall socket. You don't rebuild the power plant to run a new appliance — you plug in. Likewise, a translation app, a customer-service bot, or your company's internal tool just “plugs into” an LLM: it sends a prompt, gets text back. The prompt techniques from today are exactly what those apps send through the plug.

Writing assistants | **Customer-service bots** | **In-app search & summaries** | **Everyday AI features**

When an app advertises “AI-powered,” this plug is usually what it means.

What actually crosses the plug (no code required)

When you tap “summarize” in a notes app, one envelope travels through the plug. You now understand every item inside it:

The hidden brief

system prompt

“You are a note-summarizing assistant. Be concise, keep the user's headings, never invent content…” — a role + rules, written once by the app's team.

Your content

the user message

The note you selected — pasted in as tokens, filling part of the desk.

The settings

parameters

Low temperature for faithful summaries · a length cap · maybe a format request. The dials, chosen for you.

“AI-powered” apps are, at heart, well-crafted prompts on a plug. After today, you could sketch that envelope yourself.

USING IT WISELY

A great first-draft machine — not an oracle

Lean on it for

- First drafts of anything — emails, plans, outlines, code
- Transforming: summarize, translate, reformat, change tone
- Brainstorming and “give me ten options” thinking
- Explaining concepts at exactly your level

Double-check when

- Specific facts, numbers, dates, quotes, citations
- Anything recent — remember the frozen-in-time limit
- Legal, medical, financial stakes — expert territory
- It's going out under YOUR name

The productive mindset: treat it like a brilliant, tireless junior colleague — enormous output, always worth a final review.

PART 2 RECAP

Your steering toolkit

Zero-shot

Just ask — enough for common, well-described tasks.

Few-shot

Show 2–5 examples when the format matters more than the words.

Role

“You are a…” chooses the voice and the priorities.

Context

Audience, goal, background, constraints — the stranger test.

Dials

Tone, length, format in the prompt; temperature in settings.

APIs

The plug — same model, software-shaped door.

THE WHOLE DAY IN TWO SENTENCES

AI learned to read by learning where to look.

And because its only interface is language, the quality of your words now decides the quality of its work.

- This week: pick one real task and rewrite its prompt with Role · Context · Task · Format.
- Notice attention at work: watch how a chatbot uses details from earlier in your conversation.
- Next session: putting these models to work in real workflows.

Questions?